
Documentação de Projeto

para o sistema

EventHub – Sistema de Vendas e Gerenciamento de Eventos

Versão 1.0

Projeto de sistema elaborado por Felipe Giannetti Fontenelle
como parte da disciplina **Projeto de Software**.

08/06/2026

Tabela de Conteúdo

1. Introdução	1
2. Modelos de Usuário e Requisitos	1
2.1 Descrição de Atores	1
2.2 Modelo de Casos de Uso	2
2.3 Diagrama de Sequência do Sistema	3
3. Modelos de Projeto	6
3.1 Arquitetura	6
3.2 Diagrama de Componentes e Implantação	6
3.3 Diagrama de Classes	7
3.4 Diagramas de Sequência	8
3.5 Diagramas de Comunicação	9
3.6 Diagramas de Estados	10
4. Modelos de Dados	11

Histórico de Revisões

Nome	Data	Razões para Mudança	Versão
Felipe Giannetti Fontenelle	09/06/20 26	Criação inicial da documentação do sistema EventHub.	1.0

1. Introdução

O EventHub é uma plataforma fictícia para gestão e venda de ingressos online, inspirada em sistemas de bilheteria digital como Sympla e Ingresso.com. O sistema tem como objetivo centralizar todo o ciclo de vida de um evento: desde o cadastro pelo organizador, passando pela venda de ingressos ao participante, processamento de pagamento, emissão de ticket digital com QR Code único, até a validação de entrada no dia do evento.

O problema resolvido pela plataforma é a gestão descentralizada de eventos presenciais, que frequentemente envolve processos manuais de venda, controle de capacidade e check-in. O EventHub automatiza esses processos, garantindo rastreabilidade completa de cada ingresso e segurança no acesso ao evento.

A plataforma atende três perfis principais de usuário: o Organizador, que cria e gerencia eventos e lotes de ingressos; o Participante, que compra ingressos e recebe seus tickets digitais; e o Administrador, que modera a plataforma. Além desses, integra-se com um Gateway de Pagamento fictício para aprovação de transações e com um Serviço de QR Code para geração dos tickets.

Do ponto de vista técnico, a aplicação é desenvolvida com React 19 + TypeScript no front-end, Java 17 + Spring Boot 3.3.5 no back-end, PostgreSQL 16 como banco de dados relacional e Docker + GitHub Actions para infraestrutura e CI/CD.

Este documento agrega os modelos de domínio, requisitos, arquitetura, componentes, classes, comportamento dinâmico, estados e dados necessários para projetar a aplicação.

2. Modelos de Usuário e Requisitos

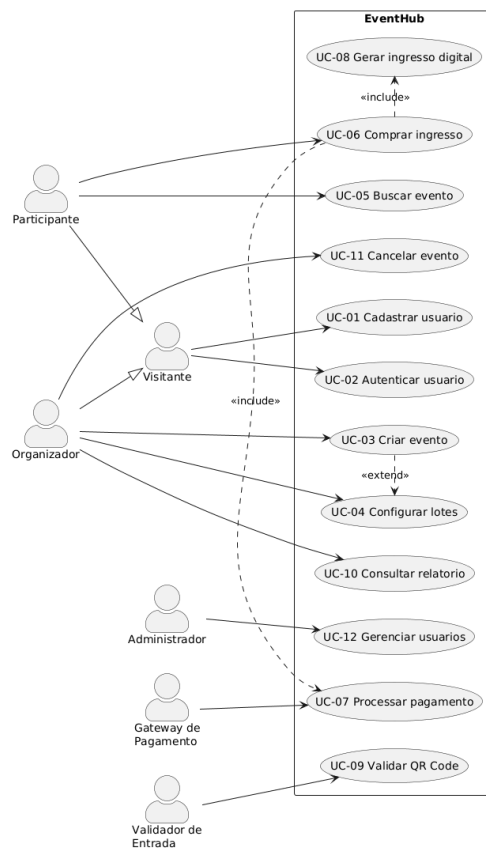
2.1 Descrição de Atores

Ator	Descrição
Visitante	Usuário não autenticado que visualiza eventos públicos e pode se cadastrar na plataforma.
Participante	Usuário cadastrado que pesquisa eventos, compra ingressos e acessa seus tickets digitais.
Organizador	Usuário responsável por criar eventos, configurar lotes de ingressos e acompanhar relatórios de vendas.
Administrador	Usuário interno que gerencia categorias, modera usuários e eventos na plataforma.
Gateway de Pagamento	Sistema externo fictício responsável por processar e aprovar pagamentos.

Ator	Descrição
Validador de Entrada	Pessoa ou dispositivo responsável por realizar o check-in validando QR Codes na entrada do evento.

2.2 Modelo de Casos de Uso

O diagrama de casos de uso apresenta as funcionalidades do sistema sob a perspectiva dos atores externos. Os casos de uso foram identificados a partir das necessidades dos três perfis principais, Organizador, Participante e Administrador, e das integrações com sistemas externos (Gateway de Pagamento e Validador de Entrada). A herança entre atores indica que Participante e Organizador herdam as capacidades básicas do Visitante (cadastro e autenticação). O relacionamento <<include>> entre UC-06, UC-07 e UC-08 indica que o fluxo de compra de ingresso obrigatoriamente inclui o processamento do pagamento e a geração do ingresso digital.



ID	Caso de Uso	Ator Principal	Descrição Resumida
UC-01	Cadastrar usuário	Visitante	O visitante preenche o formulário de cadastro e cria uma conta na plataforma.
UC-02	Autenticar usuário	Visitante	O usuário realiza login com e-mail e senha, recebendo um token JWT.

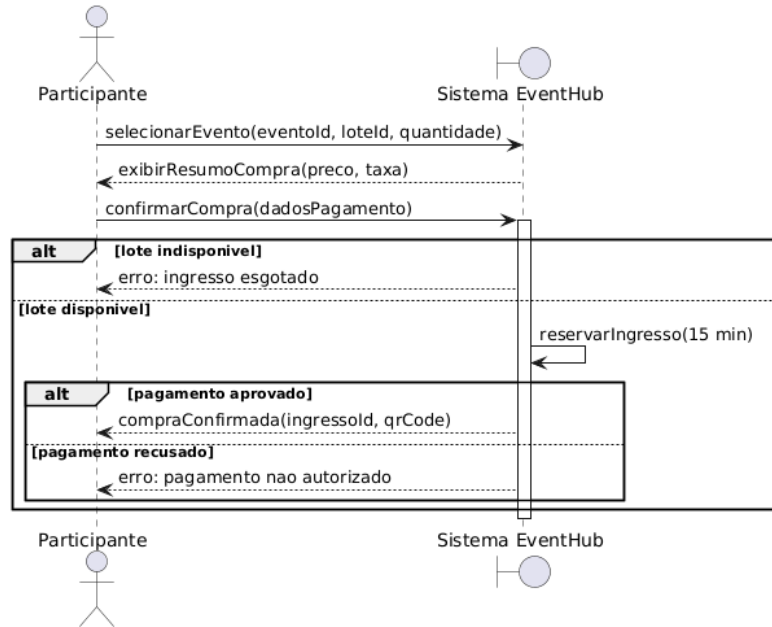
ID	Caso de Uso	Ator Principal	Descrição Resumida
UC-03	Criar evento	Organizador	O organizador cadastra um novo evento com título, data, local e capacidade.
UC-04	Configurar lotes de ingresso	Organizador	O organizador define tipos (Inteira, Meia, VIP, Cortesia), preços e períodos de venda.
UC-05	Buscar evento	Participante	O participante pesquisa eventos por nome, categoria ou data.
UC-06	Comprar ingresso	Participante	O participante seleciona um evento e lote, iniciando o fluxo de compra com reserva de 15 minutos.
UC-07	Processar pagamento	Gateway de Pagamento	O gateway recebe os dados da compra e retorna aprovação ou recusa.
UC-08	Gerar ingresso digital	Sistema	Após pagamento aprovado, o sistema gera um QR Code único e emite o ingresso digital.
UC-09	Validar ingresso por QR Code	Validador de Entrada	O validador lê o QR Code do ingresso e o sistema registra o check-in (uso único).
UC-10	Consultar relatório de vendas	Organizador	O organizador visualiza quantidade vendida, receita e taxa de serviço por evento.
UC-11	Cancelar evento	Organizador	O organizador cancela um evento publicado, disparando o processo de reembolso.
UC-12	Gerenciar usuários	Administrador	O administrador ativa, desativa ou altera o perfil de usuários na plataforma.

2.3 Diagrama de Sequência do Sistema

Os diagramas de sequência do sistema tratam o EventHub como uma caixa-preta, focando na interação entre os atores externos e o sistema como um todo. Três casos de uso centrais foram selecionados por representarem os fluxos de maior valor e complexidade da plataforma.

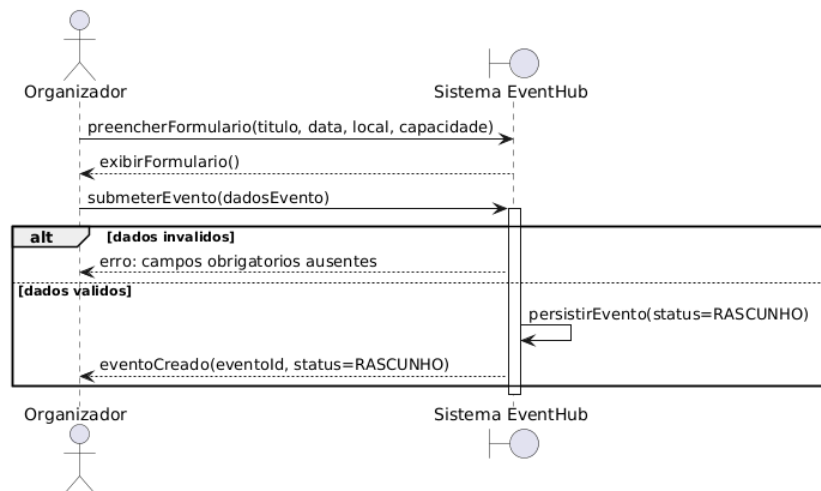
Sequência - UC-06: Comprar Ingresso

O fluxo inicia quando o Participante seleciona um evento e lote. O sistema verifica disponibilidade e abre uma reserva temporária de 15 minutos. O Participante confirma a compra com os dados de pagamento. O sistema processa o pagamento via Gateway externo: se aprovado, gera o QR Code único e emite o ingresso digital; se recusado, cancela a reserva e notifica o usuário.



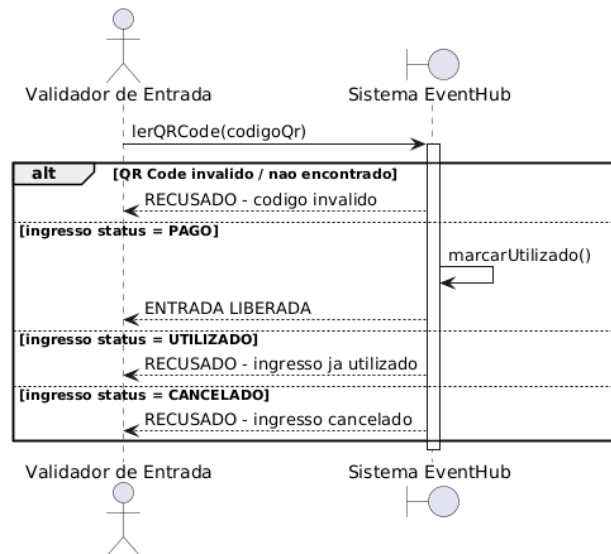
Sequência - UC-03: Criar Evento

O fluxo inicia quando o Organizador preenche o formulário com os dados do evento. O sistema valida os campos obrigatórios (título, data, local, capacidade). Se os dados forem válidos, o evento é persistido com status RASCUNHO, aguardando a configuração dos lotes de ingresso para ser publicado. Se inválidos, o sistema retorna mensagem de erro sem criar o registro.



Sequência - UC-09: Validar Ingresso por QR Code

O fluxo inicia quando o Validador de Entrada lê o QR Code de um ingresso. O sistema consulta o ingresso pelo código e avalia seu status: se PAGO, marca como UTILIZADO e libera a entrada (registro de check-in aprovado); se já UTILIZADO, recusa e registra tentativa; se CANCELADO ou inválido, recusa a entrada. O QR Code tem uso único, garantindo que cada ingresso só permita uma entrada.



Contrato	C01
Operação	comprarIngresso(eventoId, loteId, quantidade)
Referências cruzadas	UC-06, UC-07, UC-08
Pré-condições	O participante está autenticado. O evento está publicado. O lote está ativo e possui ingressos disponíveis. O timer de reserva (15 min) ainda não expirou.
Pós-condições	Uma Compra foi criada com status AGUARDANDO_PAGAMENTO. O pagamento foi processado e aprovado. Um QR Code único foi gerado. O Ingresso foi persistido com status PAGO.

Contrato	C02
Operação	criarEvento(dadosEvento)
Referências cruzadas	UC-03
Pré-condições	O organizador está autenticado. Os dados obrigatórios (título, data, capacidade, endereço) estão preenchidos.
Pós-condições	Um Evento foi criado e persistido com status RASCUNHO. O endereço foi associado ao evento.

Contrato	C03
Operação	validarIngresso(codigoQr)
Referências cruzadas	UC-09
Pré-condições	O validador está autenticado. O código QR informado corresponde a um ingresso existente no sistema.

Pós-condições	Se o ingresso estava com status PAGO, seu status foi alterado para UTILIZADO e um registro de CheckIn com resultado APROVADO foi persistido. Se o ingresso já estava UTILIZADO ou CANCELADO, nenhuma alteração de status ocorre e o CheckIn é registrado com resultado RECUSADO.
----------------------	--

3. Modelos de Projeto

3.1 Arquitetura

A arquitetura do EventHub segue o padrão de camadas (Layered Architecture) com separação clara entre interface de usuário, API, serviços de negócio, persistência e integrações externas. O front-end React (SPA) se comunica exclusivamente com o back-end via API REST sobre HTTPS, nunca acessando o banco de dados diretamente.

No back-end, cada domínio do sistema é isolado em um módulo independente: Auth, Eventos, Ingressos, Compras, Pagamentos, CheckIn e Relatórios. Esses módulos se comunicam internamente via chamadas de serviço (Service Layer) e externamente expõem endpoints REST. As integrações com serviços externos, Gateway de Pagamento, Serviço de E-mail e Serviço de QR Code, são abstraídas por interfaces, permitindo substituição sem impacto nas regras de negócio.

Os padrões adotados são: Repository Pattern (isolamento da camada de dados), Service Layer (centralização das regras de negócio), DTO (transferência de dados entre camadas), Strategy (intercambialidade de métodos de pagamento) e Observer (notificações por e-mail disparadas por eventos de domínio).

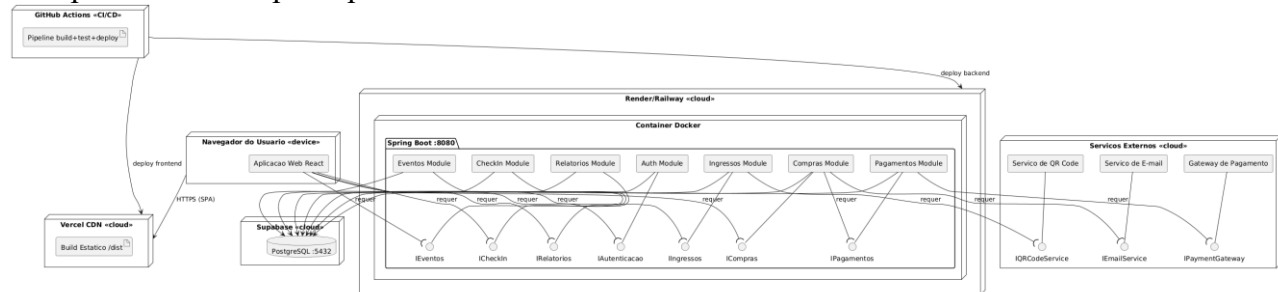
A arquitetura monolítica modular foi escolhida pela adequação ao escopo do projeto. Em um cenário de alta escala, os módulos de Pagamentos e CheckIn poderiam ser extraídos como microsserviços independentes.

3.2 Diagrama de Componentes e Implantação.

O diagrama de componentes detalha a estrutura interna do sistema e as dependências entre seus módulos. As interfaces fornecidas (—●, lollipop) representam contratos que um componente expõe para outros consumirem. As interfaces requeridas (—○, socket) representam dependências que um componente precisa de outro para funcionar.

A Aplicação Web React consome as interfaces fornecidas pelos módulos do back-end (IAutenticacao, IEventos, IIngressos, ICompras, ICheckIn, IRelatorios). O módulo de Compras requer a interface de Pagamentos (IPagamentos) e de E-mail (IEmailService); o módulo de Pagamentos requer a interface do Gateway externo (IPaymentGateway); o módulo de Ingressos requer o serviço de geração de QR Code (IQRCodeService). Todos os módulos do back-end acessam o PostgreSQL diretamente.

O diagrama de implantação mostra a topologia física da aplicação em produção: o front-end é distribuído como build estático via CDN Vercel; a API Spring Boot roda em contêiner Docker hospedado no Render/Railway; o banco PostgreSQL é provisionado em nuvem (Supabase). O pipeline de CI/CD no GitHub Actions automatiza o build, testes e deploy em ambos os ambientes a cada push na branch principal.



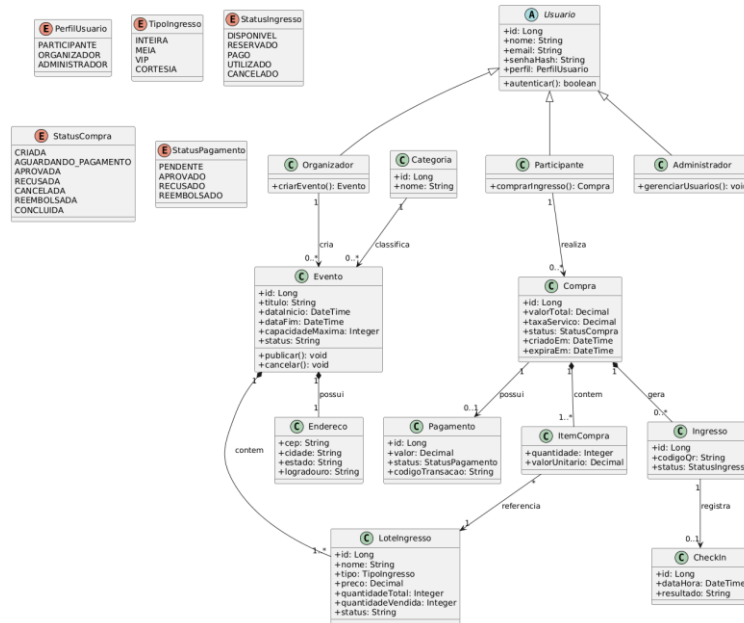
3.3 Diagrama de Classes

O diagrama de classes apresenta o modelo de domínio do EventHub com suas entidades, atributos, métodos e relacionamentos. A classe abstrata `Usuario` é especializada em três perfis concretos: `Participante`, `Organizador` e `Administrador`, utilizando herança para compartilhar atributos comuns (`id`, `nome`, `email`, `senhaHash`).

O `Organizador` cria `Eventos`, que possuem um `Endereço` (composição) e são classificados por uma `Categoria` (associação). Cada `Evento` pode ter múltiplos `LoteIngressos`, que definem tipo (`Inteira`, `Meia`, `VIP`, `Cortesia`), preço e período de venda.

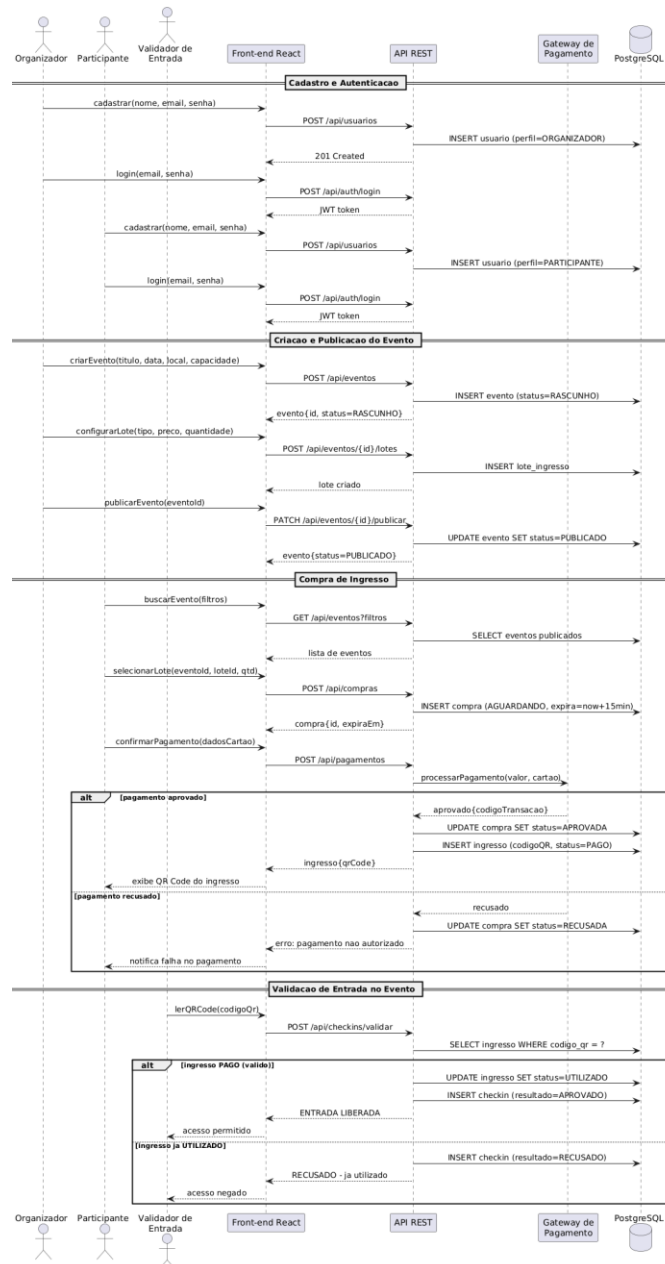
O `Participante` realiza `Compras`, compostas de `ItensCompra` que referenciam `LoteIngressos`. Cada `Compra` possui um `Pagamento` associado e gera um ou mais `Ingressos` digitais. Cada `Ingresso` pode registrar um `CheckIn` no momento da entrada no evento.

O ciclo de vida dos objetos é controlado por enumeradores: `StatusIngresso` (`DISPONIVEL` → `RESERVADO` → `PAGO` → `UTILIZADO` / `CANCELADO`) e `StatusCompra` (`CRIADA` → `AGUARDANDO_PAGAMENTO` → `APROVADA` → `CONCLUIDA` / demais estados de falha).



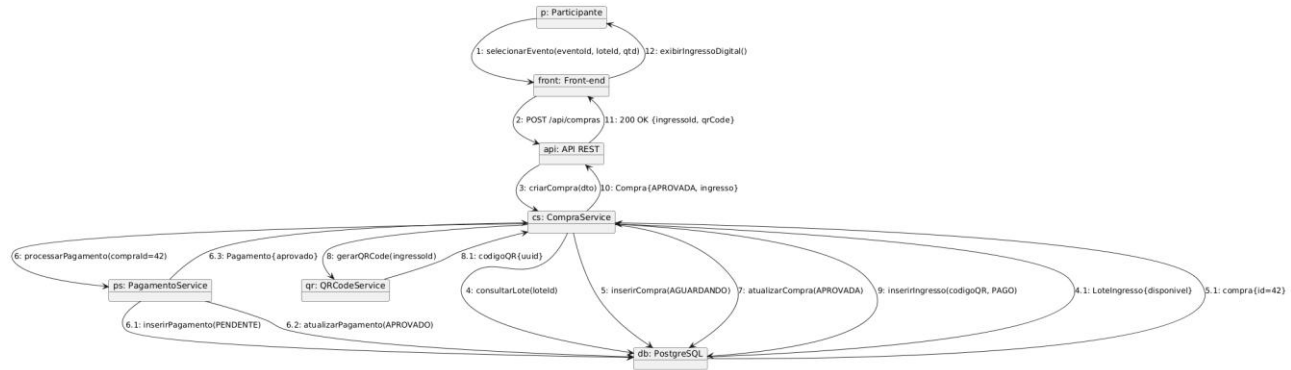
3.4 Diagramas de Sequência

O diagrama de sequência geral apresenta o fluxo completo de uso da plataforma EventHub, desde o cadastro dos usuários até a validação de ingresso no dia do evento. O objetivo é oferecer uma visão integrada de como os atores interagem com o sistema ao longo do ciclo de vida completo de um evento, conectando os casos de uso UC-01, UC-02, UC-03, UC-04, UC-06, UC-07, UC-08 e UC-09 em uma única narrativa.



3.5 Diagramas de Comunicação

O diagrama de comunicação complementa os diagramas de sequência ao evidenciar as ligações estruturais entre os objetos, numerando as mensagens para indicar a ordem de execução. O foco é mostrar quais objetos colaboram entre si e a topologia dessa colaboração, e não apenas a linha do tempo das mensagens. O fluxo modelado é o de Comprar Ingresso, por ser o mais rico em interações entre objetos.

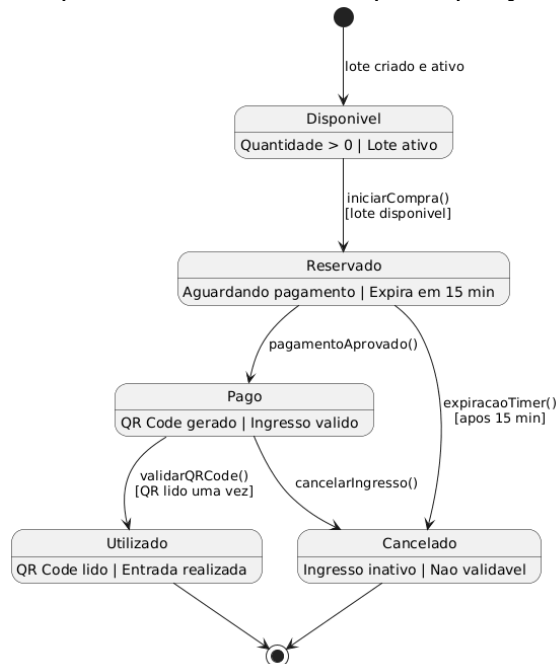


3.6 Diagramas de Estados

Os diagramas de estados modelam o ciclo de vida dos objetos mais relevantes do sistema, descrevendo os estados possíveis, as transições entre eles e as condições ou eventos que as disparam. Dois objetos foram selecionados por possuírem ciclos de vida complexos e interdependentes: o Ingresso e a Compra.

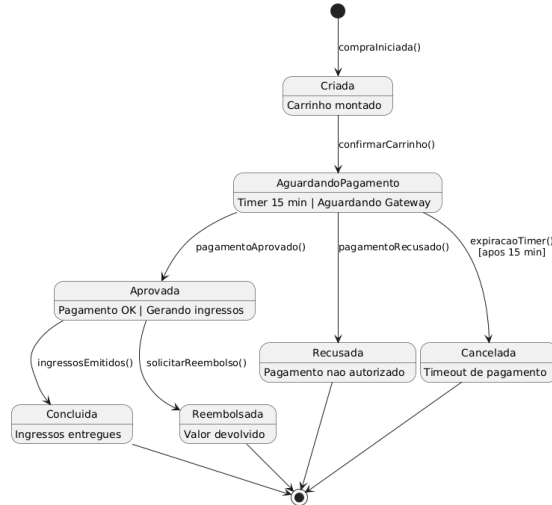
Estados do Ingresso

O Ingresso nasce no estado DISPONIVEL quando o lote é criado. Ao iniciar uma compra, entra em RESERVADO por até 15 minutos. Se o pagamento for aprovado dentro desse prazo, avança para PAGO e um QR Code é gerado. Na validação da entrada, avança para UTILIZADO (uso único). A qualquer ponto anterior ao uso, pode ser CANCELADO por expiração do timer ou reembolso.



Estados da Compra

A Compra é criada no estado CRIADA ao montar o carrinho. Após confirmação, avança para AGUARDANDO_PAGAMENTO, onde aguarda resposta do Gateway em até 15 minutos. O resultado do Gateway determina os caminhos: APROVADA (pagamento aceito) → CONCLUIDA (ingressos emitidos); ou RECUSADA (gateway recusou); ou CANCELADA (timeout). De APROVADA, ainda é possível transitar para REEMBOLSADA em caso de cancelamento posterior.



4. Modelos de Dados

O modelo de dados do EventHub foi projetado para refletir diretamente o modelo de domínio descrito no Diagrama de Classes. Cada entidade principal corresponde a uma tabela relacional, com relacionamentos implementados via chaves estrangeiras. A estratégia de mapeamento adota herança de tabela única para a entidade Usuario (campo perfil discrimina o tipo), composição via FK obrigatória para Endereco e Pagamento, e associação via FK para os demais relacionamentos.

O EventHub utiliza PostgreSQL 16 como banco de dados relacional. O mapeamento objeto-relacional é realizado com JPA/Hibernate 6, onde cada entidade do domínio corresponde a uma tabela. Relacionamentos são mapeados por chaves estrangeiras; enumeradores são persistidos como strings (EnumType.STRING). O schema é gerenciado automaticamente pelo Hibernate em ambiente de desenvolvimento.

Tabela	Campos principais
usuarios	id, nome, email, senha_hash, perfil, status, criado_em
categorias	id, nome, descricao
eventos	id, organizador_id, categoria_id, titulo, descricao, data_inicio, data_fim, capacidade_maxima, status
enderecos	id, evento_id, cep, cidade, estado, bairro, logradouro, numero
lotes_ingresso	id, evento_id, nome, tipo, preco, quantidade_total, quantidade_vendida, inicio_venda, fim_venda, status
compras	id, participante_id, valor_total, taxa_servico, status, criado_em, expira_em

Tabela	Campos principais
itens_compra	id, compra_id, lote_id, quantidade, valor_unitario
ingressos	id, compra_id, lote_id, codigo_qr, status, utilizado_em
pagamentos	id, compra_id, metodo, valor, status, codigo_transacao
checkins	id, ingresso_id, validador_id, data_hora, resultado

